

Homework 04 - Java Class, Object And Methods

Topics Covered

- Class and object
- Fields/instance variables
- Reference variables
- Creating objects using `new`
- Accessing fields from another class
- Dot operator
- Multiple objects
- Methods
- Parameters and arguments
- `void` and return values
- Local variables and instance variables

Constructors are not included in this homework.

Instructions

1. Write every program in a separate Java file.
2. Use meaningful class, object, field, and method names.
3. Create objects using `new`.
4. Access instance fields and methods using the reference variable.
5. Do not use constructors in any solution.
6. Write the expected output before running each program.

Part A: Short Answer Questions

1. What is a class?
2. What is an object?
3. Why is a class called a blueprint?
4. What is a field or instance variable?
5. What is a reference variable?
6. What does the `new` keyword do?
7. What is the dot operator used for?
8. Can one class be used to create multiple objects?
9. Does each object have a separate copy of its instance fields?
10. What is a method?
11. Why do we use methods?
12. What is the difference between method declaration and method call?
13. What is a parameter?
14. What is an argument?

15. What does `void` mean?
16. What does the `return` statement do?
17. What is a local variable?
18. What is the difference between a local variable and an instance variable?
19. What happens when two reference variables point to the same object?
20. What happens when we access a field or method using a `null` reference?

Part B: Identify The Parts

Study the code:

```
class Student {
    String name;
    int marks;

    void displayDetails() {
        System.out.println(name);
        System.out.println(marks);
    }
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student();
        student.name = "Rahul";
        student.marks = 85;
        student.displayDetails();
    }
}
```

Identify:

1. Class name
2. Reference variable
3. Object-creation expression
4. Instance fields
5. Method name
6. Method call
7. Uses of the dot operator

Part C: Find The Output

Question 1

```
class Student {
    String name;
    int marks;
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student();

        System.out.println(student.name);
        System.out.println(student.marks);
    }
}
```

Question 2

```
class Car {
    String color;
    int speed;
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();

        car.color = "Red";
        car.speed = 120;

        System.out.println(car.color);
        System.out.println(car.speed);
    }
}
```

Question 3

```
class Student {
    String name;
}

public class Main {
    public static void main(String[] args) {
        Student first = new Student();
        Student second = new Student();

        first.name = "Rahul";
        second.name = "Priya";

        first.name = "Amit";

        System.out.println(first.name);
        System.out.println(second.name);
    }
}
```

Question 4

```
class Student {
    String name;
}

public class Main {
    public static void main(String[] args) {
        Student first = new Student();
        first.name = "Rahul";

        Student second = first;
        second.name = "Amit";

        System.out.println(first.name);
        System.out.println(second.name);
    }
}
```

Question 5

```
class Counter {
    int count;

    void increase() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        Counter counter = new Counter();

        counter.increase();
        counter.increase();
        counter.increase();

        System.out.println(counter.count);
    }
}
```

Question 6

```
class Calculator {
    void add(int a, int b) {
        System.out.println(a + b);
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        calculator.add(20, 30);
    }
}
```

Question 7

```
class Calculator {
    int multiply(int a, int b) {
        return a * b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calculator = new Calculator();
        int result = calculator.multiply(4, 5);

        System.out.println(result);
    }
}
```

Question 8

```
class BankAccount {
    double balance;

    void deposit(double amount) {
        balance += amount;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
        account.balance = 1000;

        account.deposit(500);
        account.deposit(250);

        System.out.println(account.balance);
    }
}
```

Part D: Find And Correct The Error

Question 1

```
class Student {
    String name;
}

public class Main {
    public static void main(String[] args) {
        Student student;
        student.name = "Rahul";
    }
}
```

Question 2

```
class Car {
    String color;
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        color = "Blue";

        System.out.println(car.color);
    }
}
```

Question 3

```
class Calculator {
    int add(int a, int b) {
        int result = a + b;
    }
}
```

Question 4

```
class Calculator {
    void add(int a, int b) {
        return a + b;
    }
}
```

Question 5

```
class Student {
    void display(String name, int age) {
        System.out.println(name);
        System.out.println(age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student student = new Student();
        student.display(20, "Rahul");
    }
}
```

Question 6

```
class Student {
    String name;
}

public class Main {
    public static void main(String[] args) {
        Student student = null;
        System.out.println(student.name);
    }
}
```

Explain the runtime problem as well as correcting the program.

Part E: Class And Object Programs

Program 1: Student Details

Create a `Student` class containing:

```
String name;
int rollNumber;
int age;
int marks;
```

In another class containing `main` :

1. Create a `Student` object.
2. Assign field values using the dot operator.
3. Print all details.

Program 2: Multiple Students

Create three `Student` objects with different names, roll numbers, and marks.

Print the details of all three objects.

Program 3: Car Details

Create a `Car` class containing:

```
String brand;
String model;
String color;
double price;
```

Create two objects and print their details.

Program 4: Employee Details

Create an `Employee` class containing:


```
int employeeId;  
String employeeName;  
double monthlySalary;
```

Create an object and calculate annual salary inside `main` .

Program 5: Product Details

Create a `Product` class containing:

```
String name;  
double price;  
int quantity;
```

Create an object and calculate total price.

Part F: Method Programs

Program 1: Student Method

Add a method to the Student class:

```
void displayDetails()
```

The method should print all fields of the current object.

Program 2: Calculator

Create a `Calculator` class with methods:

```
int add(int a, int b)  
int subtract(int a, int b)  
int multiply(int a, int b)  
double divide(double a, double b)
```

Create an object and call every method.

Program 3: Rectangle

Create a `Rectangle` class containing:

```
double length;  
double width;
```

Add methods:

```
double calculateArea()  
double calculatePerimeter()
```

Program 4: Bank Account

Create a `BankAccount` class containing:

```
String accountHolder;  
double balance;
```

Add methods:

```
void deposit(double amount)  
void withdraw(double amount)  
void displayBalance()
```

Do not allow withdrawal when the amount is greater than the balance.

Program 5: Product Service

Add a method:

```
double calculateTotal()
```

It should return:

```
price * quantity
```

Program 6: Pass Or Fail

Create a `Student` class with a `marks` field.

Add:

```
String getResult()
```

Return `"Pass"` when marks are at least 40; otherwise return `"Fail"`.

Part G: Method-Type Practice

Write one example for each type:

1. No parameter and no return value
2. Parameters and no return value
3. No parameter and a return value
4. Parameters and a return value

Use a `Calculator`, `Student`, or `Message` class.

Part H: Combined Challenges

Challenge 1: Highest Marks

Create three `Student` objects.

Use `if-else` inside `main` to print the student with the highest marks.

Challenge 2: Salary Update

Create an `Employee` class with:

```
String name;  
double salary;
```

Add:

```
void addBonus(double percentage)
```

The method should update the object's salary.

Challenge 3: Inventory

Create a Product class with:

```
String name;  
int stock;
```

Add methods:

```
void addStock(int quantity)  
void sell(int quantity)
```

Do not allow sales greater than available stock.

Challenge 4: Object Array

Create a Student array that stores three Student objects.

Assign fields without constructors and use a loop to print every student's details.

Hint:

```
Student[] students = new Student[3];  
  
students[0] = new Student();  
students[1] = new Student();  
students[2] = new Student();
```

Challenge 5: Array Parameter

Create a class `NumberService`.

Add:

```
int calculateSum(int[] numbers)
```

Call it by passing an integer array.

Part I: Oral Revision

1. What is the difference between a class and an object?
2. Which keyword creates an object?
3. What is stored in a reference variable?
4. How do we access a field from another class?

5. What is the difference between a field and a local variable?
6. What is the difference between a parameter and an argument?
7. What does `void` mean?
8. Why do methods use `return` ?
9. Can two reference variables point to the same object?
10. What exception occurs when a `null` reference is used?

Submission Checklist

- Constructors have not been used.
- Every object is created using `new` .
- Fields are accessed using the correct reference.
- Each method has a meaningful name.
- Parameter order and data types are correct.
- Non-void methods return a compatible value.
- Instance and local variables are used correctly.
- Output is written clearly.

